DV INTERVIEW · GLANCE SHEET 5

Bursts: Length, Size & Type

Three fields decide every address in a burst. Master them and you can compute any beat, lane and strobe on a whiteboard.

```
beats = AxLEN + 1
bytes/beat = 2^AxSIZE
max bytes = (LEN+1) × 2^SIZE
```

AXLEN · HOW MANY

Burst length

Number of data beats − 1

Type	AXI3	AXI4
INCR	1–16	**1–256**
WRAP	2, 4, 8, 16	2, 4, 8, 16
FIXED	1–16	1–16
Field width	4 bits	8 bits

`AxLEN = 0` means **1** beat. `AxLEN = 8'hFF` means 256 beats.

AXSIZE · HOW WIDE

Burst size

Bytes in each beat = 2^SIZE

SIZE	Bytes	SIZE	Bytes
000	1	100	16
001	2	101	32
010	4	110	64
011	8	111	128

2^SIZE must be **≤ data bus width**. Smaller than the bus = **narrow transfer**.

AXBURST · WHICH PATTERN

Burst type

How the address moves beat to beat

Code	Type	Address	Use
00	**FIXED**	Same each beat	FIFO, data port
01	**INCR**	+2^SIZE each beat	Normal memory
10	**WRAP**	+2^SIZE, wraps	Cache line fill
11	—	Reserved	Error

INCR is the **default** and the most common. AXI-Lite has no bursts at all.

ƒ Address & byte-lane formulas (works for every burst)

DEFINITIONS

```
N_bytes   = 2 ** AxSIZE              // bytes per beat
Beats     = AxLEN + 1
Aligned   = (Start / N_bytes) * N_bytes   // integer divide
Bus_bytes = DATA_WIDTH / 8
```

ADDRESS OF BEAT N (N = 1 … BEATS)

```
FIXED : Addr[n] = Start
INCR  : Addr[1] = Start
        Addr[n] = Aligned + (n-1) * N_bytes      // n ≥ 2
WRAP  : Lower   = (Start / (N_bytes*Beats)) * (N_bytes*Beats)
        Upper   = Lower + N_bytes*Beats
        Addr[n] = INCR formula, but if it hits Upper → Lower
```

BYTE LANES OF BEAT N (FOR WSTRB)

```
Low_lane  = Addr[n] % Bus_bytes
High_lane = (Aligned_n % Bus_bytes) + N_bytes - 1
// Aligned_n = Addr[n] rounded down to N_bytes
// first beat may start mid-way (unaligned)
```

TOTAL BYTES ACTUALLY MOVED

```
Total = Beats * N_bytes - (Start % N_bytes)
// the subtracted part is the unaligned head
```

Only the first beat can be unaligned. From beat 2 onward INCR addresses are always multiples of N_bytes.

I INCR burst in detail — 4 worked examples

EX 1 · ALIGNED, FULL WIDTH · 32-BIT BUS · START 0X1000 · LEN 7 · SIZE 010

`0x1000` `0x1004` `0x1008` `0x100C` `0x1010` `0x1014` `0x1018` `0x101C`

8 beats × 4 B = **32 bytes**. Every WSTRB = `1111`. Last byte 0x101F.

EX 2 · NARROW · 64-BIT BUS (8 LANES) · START 0X04 · LEN 3 · SIZE 010 (4 B)

0x04	**7**	**6**	**5**	**4**	3	2	1	0	`WSTRB 0xF0`
0x08	7	6	5	4	**3**	**2**	**1**	**0**	`WSTRB 0x0F`
0x0C	**7**	**6**	**5**	**4**	3	2	1	0	`WSTRB 0xF0`
0x10	7	6	5	4	**3**	**2**	**1**	**0**	`WSTRB 0x0F`

The 4-byte window **ping-pongs** between upper and lower halves of the bus.

EX 3 · UNALIGNED · 32-BIT BUS · START 0X07 · LEN 2 · SIZE 010

0x07	**3**	2	1	0	`1000`	1 byte
0x08	**3**	**2**	**1**	**0**	`1111`	4 bytes
0x0C	**3**	**2**	**1**	**0**	`1111`	4 bytes

Aligned = 0x04, so beat 2 = 0x04 + 4 = **0x08** (not 0x0B). Total = 3×4 − 3 = **9 bytes**.

EX 4 · THE 4 KB BOUNDARY CHECK

`0xFF0` LEN 3, SIZE 010 → 16 B → last byte `0xFFF` ✓ legal, ends exactly at boundary

`0xFF8` LEN 3, SIZE 010 → 16 B → last byte `0x1007` × crosses 0x1000

Fix: split into two bursts — 0xFF8 (LEN 1, 8 B) and 0x1000 (LEN 1, 8 B). Interconnects / DMA engines do this automatically.

Check: address bits [MSB:12] of the first byte and last byte must be equal.

F FIXED burst

32-BIT BUS · START 0X20 · LEN 3 · SIZE 010

`0x20` `0x20` `0x20` `0x20`

- Same address, same byte lanes on every beat.
- Used to push/pop a **FIFO** or a peripheral data register.
- Max 16 beats in AXI3 and AXI4.
- Rarely supported by memory slaves — cover it only if the DUT claims support.

W WRAP burst

32-BIT BUS · START 0X14 · LEN 7 (8 BEATS) · SIZE 010

Container = 8 × 4 = **32 B** → Lower = INT(0x14 / 32) × 32 = **0x00**, Upper = **0x20**.

`0x14` `0x18` `0x1C` → hits 0x20, wrap → `0x00` `0x04` `0x08` `0x0C` `0x10`

- Length must be 2, 4, 8 or 16; start must be **aligned to SIZE**.
- Total container is always a power of two, so a WRAP never crosses 4 KB.
- Why: CPU misses on word 0x14 → that word returns **first** ("critical word first"), rest of the cache line follows.
- If start is already at Lower, a WRAP looks exactly like an INCR.

✓ Burst constraints & coverage (SV)

```
class axi_txn extends uvm_sequence_item;
  rand bit [31:0] addr;
  rand bit [7:0]  len;
  rand bit [2:0]  size;
  rand bit [1:0]  burst;

  constraint c_type  { burst != 2'b11; }
  constraint c_size  { (1 << size) <= BUS_BYTES; }
  constraint c_len   {
    burst == WRAP  -> len inside {1, 3, 7, 15};
    burst == FIXED -> len < 16; }
  constraint c_wrap  { burst == WRAP -> addr % (1 << size) == 0; }
  constraint c_4k    { burst == INCR ->
    (addr % 4096) + (len + 1) * (1 << size) <= 4096; }
endclass
```

c_4k is safe (slightly conservative for unaligned starts). Use `solve burst before len` for better distribution.

COVERAGE

- **cross burst × len_bins × size** (len bins: 1, 2–15, 16, 17–255, 256).
- aligned vs unaligned start × narrow vs full width.
- INCR ending exactly at a 4 KB boundary; WRAP starting at every offset in the container.

? Burst interview Q&A

Question	Crisp answer
LEN = 15, SIZE = 3 — bytes?	16 × 8 = 128 bytes.
Max bytes in one AXI4 INCR on a 128-bit bus?	256 × 16 = 4096 — but 4 KB rule means it must start at a 4 KB-aligned address.
Why the 4 KB limit?	Smallest slave region / page; a burst must stay in one slave.
Why only 16 beats for WRAP?	It's for cache lines, which are small; power-of-2 container needed.
Can a burst stop early?	No — all LEN+1 beats must transfer; use WSTRB = 0 to skip writes.
Next address after unaligned 0x07, SIZE 4?	0x08 (aligned base 0x04 + 4).
Can SIZE be larger than the bus?	No — illegal.
Can WRAP start unaligned?	No — start must be a multiple of 2^SIZE.
WRAP 4 beats, 8 B, start 0x18?	Container 32 B → 0x18, 0x00, 0x08, 0x10.